

TRƯỜNG ĐẠI HỌC CNTT - ĐHQG TP.HCM
KHOA KHOA HỌC MÁY TÍNH

Môn học
Phân tích và thiết kế thuật toán
(Design and Analysis of Algorithms)

BÀI TẬP LỚN

Trực quan hóa thuật toán Quay lui giải bài toán Địa chỉ IP
(WeCode)

Assignment: Bài tập lớn
Họ và Tên: Đỗ Quốc Học
MSSV: 26410045
Ngày nộp bài: 07/10/2026

Mục lục

1	Sản phẩm nộp	3
2	Giới thiệu bài toán	3
3	Mô tả bài toán	3
4	Mô hình bài toán	4
4.1	Input, Output	4
4.2	Ràng buộc	4
4.3	Ví dụ	4
5	Thiết kế cấu trúc dữ liệu và thuật toán	4
5.1	Cấu trúc dữ liệu	4
5.2	Phân tích đặc trưng và ý tưởng	5
5.3	Mã giả	5
5.4	Tính đúng đắn	5
5.5	Độ phức tạp	6
6	Ví dụ minh họa các bước thực hiện	6
7	Thiết kế và xây dựng ứng dụng	7
7.1	Công nghệ	7
7.2	Chức năng	7
7.3	Hướng dẫn sử dụng bảng điều khiển (dashboard)	9
8	Thiết kế thực nghiệm	12
8.1	Cách tạo testcase	12
8.2	Phân nhóm	12
8.3	Mức độ bao phủ	13
9	Đánh giá	13
9.1	Tính đúng đắn	13
9.2	So sánh ba cách giải	14
9.3	Tính trực quan	15
9.4	Hạn chế và hướng phát triển	15
10	Quy trình thực hiện với AI	15
10.1	Công cụ	15
10.2	Phân công	16
10.3	Các bước và yêu cầu chính	16
10.4	Lỗi gặp phải và cách phát hiện, sửa	17
10.5	Kiểm chứng kết quả: ai làm gì	17
10.6	Nhận xét cá nhân	18
11	Tài liệu tham khảo	18
12	Kết luận	19
A	Cây quay lui đầy đủ của 25525511135	19

1 Sản phẩm nộp

1. Source code: thư mục `source/`

- `index.html` — ứng dụng web trực quan hóa (mở trực tiếp bằng trình duyệt, không cần cài đặt).
- `ip_wecode.py` — lời giải Python (quay lui) dùng để nộp WeCode.
- `ip_bruteforce.py` — bản vét cạn độc lập, dùng làm đáp án đối chứng.
- `gen_testcases.py`, `embed_tests.py`, `test_core.js` — sinh testcase, nhúng vào trang và kiểm thử lời thuật toán.

2. Video demo: `demo/demo_video.mp4` (3 phút 9 giây, 11 cảnh, có giọng đọc và phụ đề). Video được nén cùng toàn bộ sản phẩm thành một tệp `.rar` và nộp qua đường dẫn của cô.

3. Bộ testcase: thư mục `testcases/` (48 cặp `.in/.out`, `tests.json`, `README.md`).

4. Báo cáo: tài liệu này (`report/`).

2 Giới thiệu bài toán

Lý do chọn. Bài “Địa chỉ IP” có liên quan trực tiếp đến công việc của em, nên em nắm rõ ngữ cảnh sử dụng của bài toán và đánh giá được kết quả của ứng dụng có hợp lý hay không. Đây cũng là bài em đã làm ở Lab#01 (Backtracking) trên WeCode, nay chọn lại làm bài tập lớn cuối kì, nên em đã nắm rõ đề và tập trung vào phần trực quan hóa và kiểm thử. Về mặt thuật toán, bài thuộc Lab#01, đề ghi rõ phải dùng quay lui, kỹ thuật khác thì giảng viên không chấm. Bài có không gian trạng thái nhỏ và có cấu trúc rõ (bốn đoạn, mỗi đoạn một đến ba chữ số), nên cây quay lui đủ nhỏ để vẽ ra và theo dõi từng bước, nhưng vẫn có đủ hai loại loại bỏ nhánh khác nhau (loại ứng viên sai và cắt tỉa theo độ dài). Đây là điều em cần để so sánh “quay lui có và không cắt tỉa” với “vét cạn”.

Nguồn gốc và phần tham khảo. Tên bài, yêu cầu dùng quay lui và ví dụ 25525511135 lấy từ đề của môn trên WeCode (Lab#01), do em cung cấp cho AI dưới dạng ảnh chụp đề. Các phần còn lại **không có trong phần đề được cung cấp**, mà AI (Claude Sonnet 5.5) dựa vào kiến thức có sẵn để xây dựng: luật của một địa chỉ IPv4 (bốn đoạn, giá trị 0 đến 255, không số 0 đứng đầu), khung thuật toán quay lui, kỹ thuật cắt tỉa theo độ dài còn lại, thứ tự thử độ dài 1, 2, 3. Bài này cùng dạng với bài “Restore IP Addresses” (LeetCode 93). Để không chỉ dựa vào trí nhớ của AI, các điểm trên được đối chiếu lại với các nguồn ở mục 11, ghi rõ điều gì nguồn xác nhận và điều gì không.

Ứng dụng thực tế. Khôi phục dữ liệu bị mất ký tự phân cách (log mạng, văn bản OCR, tệp cấu hình hỏng), và là ví dụ điển hình của dạng “chia chuỗi thành các phần thỏa ràng buộc” trong phân tích cú pháp.

3 Mô tả bài toán

Bạn Bình cấu hình mạng cho một tiệm net. Do phần mềm gõ văn bản gặp lỗi, mọi dấu chấm trong các địa chỉ IP bị mất. Cho chuỗi s chỉ gồm các chữ số, hãy liệt kê mọi địa chỉ IPv4 hợp lệ có thể có bằng cách chèn đúng ba dấu chấm vào s .

Giả định về luật địa chỉ IP. Phần đề được cung cấp cho AI chỉ nói “các địa chỉ IP có thể có”, không liệt kê điều kiện của từng đoạn. Em áp dụng luật chuẩn của IPv4: bốn đoạn,

giá trị từ 0 đến 255, không có số 0 đứng đầu (trừ chính số 0). Ví dụ của đề (25525511135 cho đúng hai địa chỉ) khớp với luật này, nhưng đề không nêu luật thành lời, nên coi đây là giả định chuẩn và đã đối chiếu nguồn ở mục 11.

4 Mô hình bài toán

4.1 Input, Output

- **Input:** chuỗi s ($|s| = n$), kiểu `string`, giới hạn $1 \leq n \leq 100$ theo đề (“không quá 100 ký tự”), mỗi ký tự $s_i \in \{0, 1, \dots, 9\}$.
- **Output:** tập R gồm mọi địa chỉ IP $a_1.a_2.a_3.a_4$ thỏa các ràng buộc dưới đây, mỗi địa chỉ một dòng, thứ tự tùy ý. Nếu $R = \emptyset$ thì không in gì.

4.2 Ràng buộc

Địa chỉ $a_1.a_2.a_3.a_4$ hợp lệ khi và chỉ khi:

1. $a_1a_2a_3a_4 = s$ (ghép bốn đoạn theo thứ tự ra đúng chuỗi s , dùng hết ký tự);
2. $1 \leq |a_k| \leq 3$ với mọi k ;
3. $0 \leq \text{int}(a_k) \leq 255$;
4. a_k không có số 0 đứng đầu, trừ khi $a_k = “0”$.

4.3 Ví dụ

Input	Output	Ghi chú
25525511135	255.255.11.135, 255.255.111.35	ví dụ của đề
010010	0.10.0.10, 0.100.1.0	số 0 đứng đầu
0000	0.0.0.0	chỉ một cách
123	(không có)	ngắn hơn 4 ký tự
256256256256	(không có)	mọi đoạn > 255

5 Thiết kế cấu trúc dữ liệu và thuật toán

5.1 Cấu trúc dữ liệu

- **s:** chuỗi đầu vào, $n = |s|$.
- **parts:** ngăn xếp (stack) các đoạn đã chọn, tối đa 4 phần tử. Chiều dài **parts** chính là k , số đoạn đã chọn.
- **pos:** vị trí ký tự kế tiếp chưa dùng trong s .
- **out:** danh sách kết quả.
- (Chỉ có trong ứng dụng) **tree:** mảng nút của cây quay lui, mỗi nút lưu cha, đoạn đã chọn, độ sâu và trạng thái (đang thử, hợp lệ, bị loại, bị tẩy, là nghiệm).

5.2 Phân tích đặc trưng và ý tưởng

Một nghiệm là một cách chọn 3 vị trí cắt trong $n - 1$ khe giữa các ký tự. Mỗi đoạn chỉ có 3 độ dài khả dĩ, nên cây quyết định có độ sâu tối đa 4 và nhân tố rẽ nhánh tối đa 3, tức không quá $3^4 = 81$ lá, bất kể n . Hai nhận xét cho phép loại nhánh sớm:

1. **Loại ứng viên.** Nếu đoạn seg dài hơn 1 ký tự mà bắt đầu bằng '0', hoặc $\text{int}(seg) > 255$, thì loại. Hơn nữa mọi đoạn *dài hơn* bắt đầu từ cùng vị trí cũng sai (vẫn bắt đầu bằng '0', hoặc là số lớn hơn), nên được phép **break** cả vòng lặp độ dài.
2. **Cắt tỉa theo độ dài còn lại.** Đã chọn k đoạn, còn $\ell = n - \text{pos}$ ký tự và cần $4 - k$ đoạn nữa. Cần $\ell \geq 4 - k$ (mỗi đoạn ít nhất 1 ký tự) và $\ell \leq 3(4 - k)$ (mỗi đoạn nhiều nhất 3 ký tự). Vi phạm một trong hai thì không thể ra nghiệm.

5.3 Mã giả

Thuật toán 1 BACKTRACK(pos, parts)

Input: chuỗi s ($n = |s|$), vị trí pos , ngăn xếp $parts$

Output: ghi vào out mọi địa chỉ hợp lệ có tiền tố $parts$

```
1:  $k \leftarrow |parts|$ ;  $\ell \leftarrow n - pos$ 
2: if  $\ell < 4 - k$  or  $\ell > 3(4 - k)$  then return ▷ cắt tỉa theo độ dài
3: end if
4: if  $k = 4$  then
5:   if  $pos = n$  then thêm  $\text{join}(parts, '.')$  vào  $out$ 
6:   end if
7:   return
8: end if
9: for  $len \leftarrow 1$  to 3 do
10:  if  $pos + len > n$  then break
11:  end if
12:   $seg \leftarrow s[pos .. pos + len)$ 
13:  if  $len > 1$  and  $seg[0] = '0'$  then break
14:  end if
15:  if  $\text{int}(seg) > 255$  then break
16:  end if
17:   $parts.push(seg)$ 
18:  BACKTRACK( $pos + len, parts$ )
19:   $parts.pop()$ 
20: end for
```

Gọi BACKTRACK(0, []). Mã nguồn tương ứng nằm ở `source/ip_wecode.py` (hàm `bt`).

5.4 Tính đúng đắn

An toàn (không bỏ sót nghiệm). Cả ba chỗ loại bỏ đều chỉ loại những lựa chọn chắc chắn không thể dẫn tới nghiệm: đoạn có số 0 đầu, đoạn > 255 (và các đoạn dài hơn, như đã chứng minh ở trên), và tình huống độ dài còn lại không chia nổi cho số đoạn còn lại. *Đầy đủ (không sinh nghiệm sai).* Nghiệm chỉ được ghi khi $k = 4$ và $pos = n$, mỗi đoạn đã qua kiểm tra. *Không trùng.* Hai đường đi khác nhau trên cây có dãy độ dài đoạn khác nhau, tức vị trí chấm khác nhau, nên cho hai chuỗi khác nhau.

5.5 Độ phức tạp

Cây có độ sâu ≤ 4 , mỗi nút có ≤ 3 con, nên số nút $\leq 1 + 3 + 9 + 27 + 81 = 121$. Số này khớp với đo đạc: với chuỗi dài, quay lui không cắt tỉa thử đúng 120 đoạn (cột “QL không tỉa” của bảng 1), tức 121 nút trừ nút gốc. Mỗi nút tốn $O(1)$ (đoạn dài tối đa 3 ký tự), cộng thêm $O(n)$ để ghép mỗi nghiệm. Vậy **thời gian** $O(3^4 + |R| \cdot n) = O(n)$ **theo** n , tức hằng số theo bài toán này, và bộ nhớ $O(n)$.

Với vết cạy, số cách chọn 3 vị trí cắt là $\binom{n-1}{3} = \Theta(n^3)$, tức 156 849 cách khi $n = 100$. Quay lui có cắt tỉa gấp chuỗi quá dài thì dừng ngay tại gốc (1 nút).

6 Ví dụ minh họa các bước thực hiện

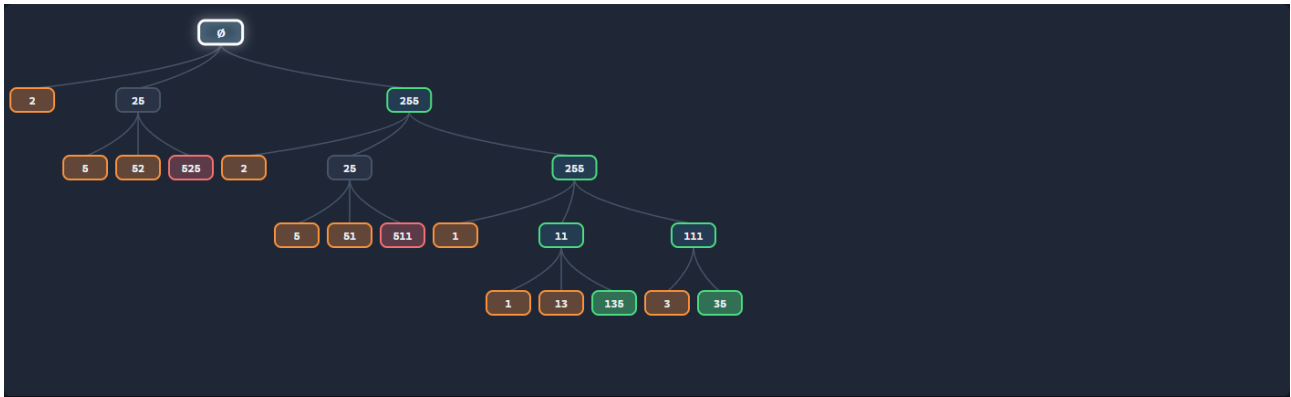
Xét $s = 25525511135$ ($n = 11$). Đây là cây quay lui thật do ứng dụng sinh ra (xuất từ `runA11`), 21 nút kể cả gốc, trạng thái tỉa độ dài bất.

Mỗi dòng dưới đây ghi k (số đoạn đã chọn sau khi thêm đoạn này), ℓ (số ký tự còn lại) và khoảng ℓ cần có để còn khả năng ra nghiệm, $[4 - k, 3(4 - k)]$. Dữ liệu lấy trực tiếp từ cây do ứng dụng sinh ra.

Nút	Đường đi	Trạng thái	k, ℓ	Cần $\ell \in$	Lý do
1	2	tỉa	1, 10	[3, 9]	$10 > 9$
2	25	hết nhánh con	1, 9	[3, 9]	hợp lệ, các con đều tỉa hoặc loại
3	25.5	tỉa	2, 8	[2, 6]	$8 > 6$
4	25.52	tỉa	2, 7	[2, 6]	$7 > 6$
5	25.525	loại	2, 6	—	$525 > 255$
6	255	có nghiệm	1, 8	[3, 9]	hợp lệ
7	255.2	tỉa	2, 7	[2, 6]	$7 > 6$
8	255.25	hết nhánh con	2, 6	[2, 6]	hợp lệ, các con đều tỉa hoặc loại
9	255.25.5	tỉa	3, 5	[1, 3]	$5 > 3$
10	255.25.51	tỉa	3, 4	[1, 3]	$4 > 3$
11	255.25.511	loại	3, 3	—	$511 > 255$
12	255.255	có nghiệm	2, 5	[2, 6]	hợp lệ
13	255.255.1	tỉa	3, 4	[1, 3]	$4 > 3$
14	255.255.11	có nghiệm	3, 3	[1, 3]	hợp lệ
15	255.255.11.1	tỉa	4, 2	[0, 0]	còn dư 2 ký tự
16	255.255.11.13	tỉa	4, 1	[0, 0]	còn dư 1 ký tự
17	255.255.11.135	ng nghiệm	4, 0	[0, 0]	dùng hết chuỗi
18	255.255.111	có nghiệm	3, 2	[1, 3]	hợp lệ
19	255.255.111.3	tỉa	4, 1	[0, 0]	còn dư 1 ký tự
20	255.255.111.35	ng nghiệm	4, 0	[0, 0]	dùng hết chuỗi

Chú thích: “tỉa” là bị cắt ngay đầu lần gọi đệ quy của nút đó bởi điều kiện độ dài. “Hết nhánh con” là nút hợp lệ nhưng mọi nút con của nó đều bị tỉa hoặc loại, nên không dẫn tới nghiệm.

Tổng hợp số liệu của lần chạy này (khớp với bảng thống kê trên giao diện): **19 nút được duyệt, 20 đoạn đã thử, 2 đoạn bị loại, 10 nhánh bị tỉa, 2 địa chỉ.**



Hình 1: Cây quay lui của 25525511135. Cam: tĩa theo độ dài. Đỏ: loại (> 255 hoặc 0 đầu). Xanh lá: nghiệm hoặc nhánh có nghiệm.

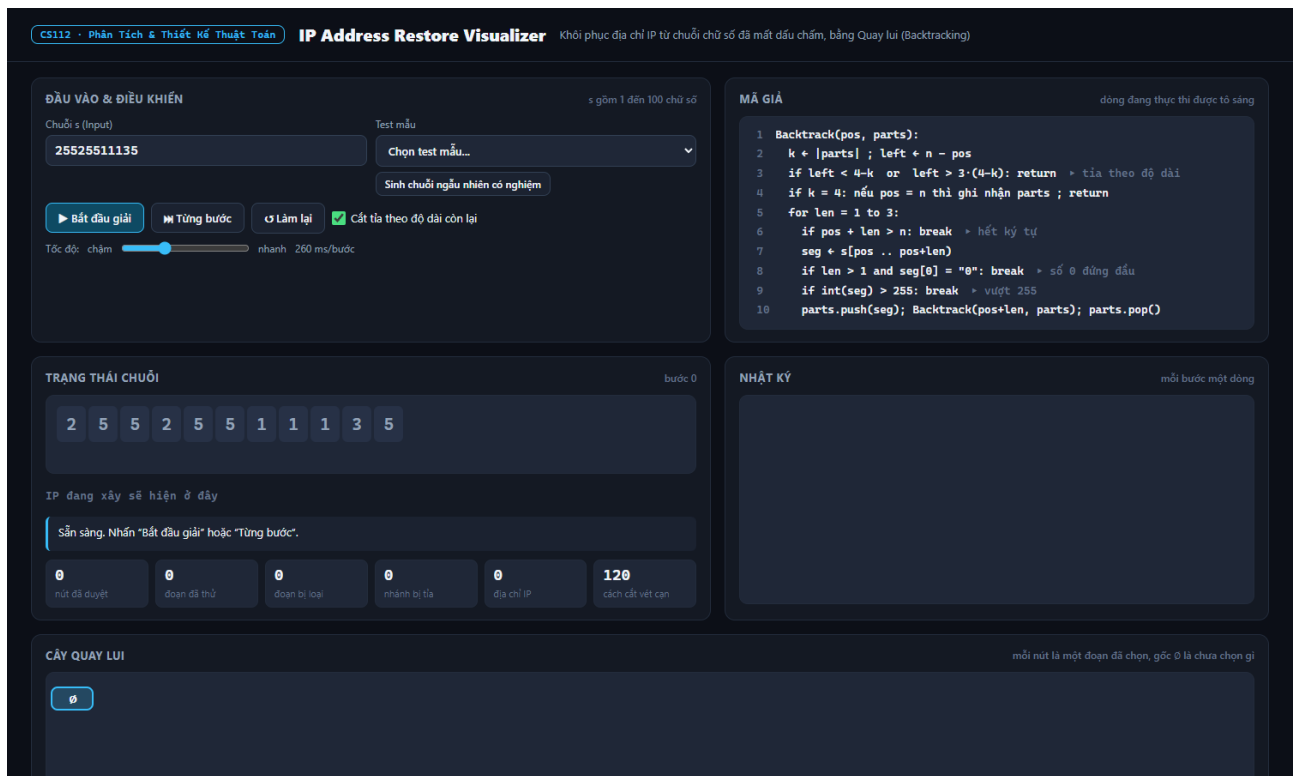
7 Thiết kế và xây dựng ứng dụng

7.1 Công nghệ

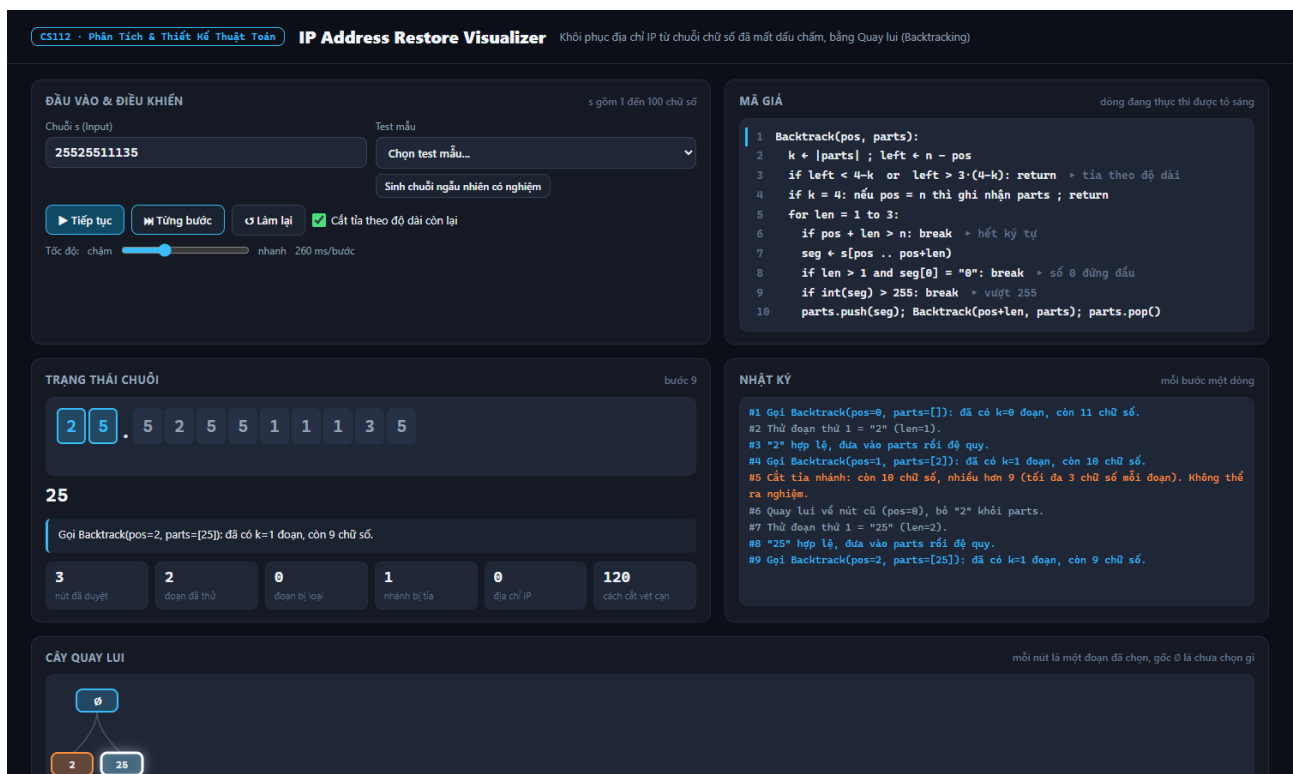
Một tệp `index.html` duy nhất (HTML, CSS, JavaScript thuần), mở bằng trình duyệt là chạy. Thuật toán viết bằng **generator function**: mỗi `yield` là một sự kiện (vào nút, thử đoạn, loại, tĩa, nhận nghiệm, quay lui). Nhờ vậy ứng dụng chạy từng bước, tạm dừng, chỉnh tốc độ mà không treo giao diện. Cây vẽ bằng SVG, tô lại sau mỗi sự kiện.

7.2 Chức năng

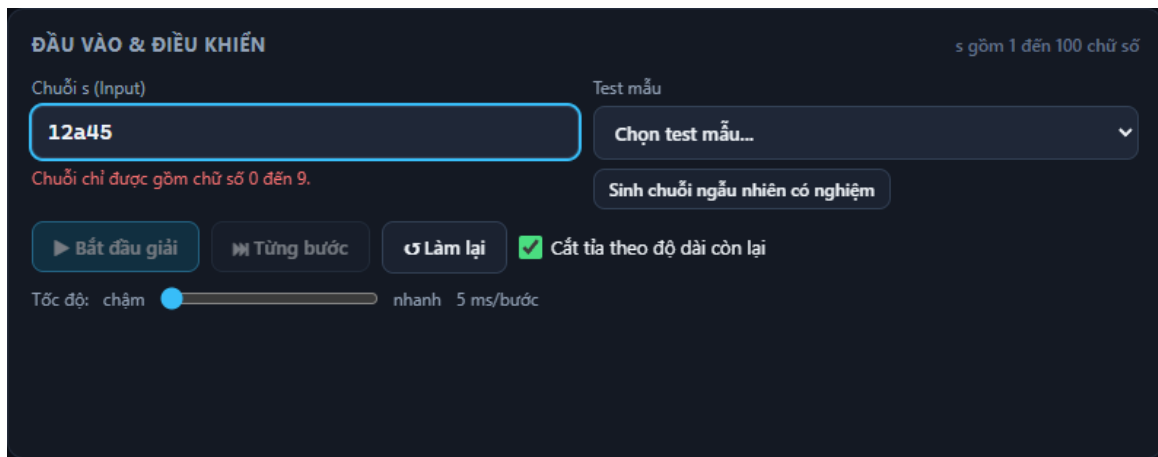
1. **Nhập input** có kiểm tra: rỗng, có ký tự không phải số, dài hơn 100 đều báo lỗi và khóa nút giải (hình 4).
2. **Chạy tự động / từng bước / làm lại**, thanh trượt tốc độ.
3. **Dải chữ số** tô màu theo bốn đoạn đã chọn, đoạn đang thử tô vàng (đỏ nếu bị loại).
4. **Mã giả** có dòng đang thực thi được tô sáng, đối chiếu từng bước với mã giả ở mục 5.
5. **Nhập ký** mỗi bước một dòng, ghi lý do.
6. **Cây quay lui** vẽ theo thời gian thực, đổi màu theo trạng thái nút.
7. **Bảng thống kê**: số nút duyệt, đoạn đã thử, đoạn bị loại, nhánh bị tĩa, số IP, số cách cắt của vết cặn.
8. **Công tắc cắt tĩa theo độ dài** để thấy cây thay đổi thế nào.
9. **So sánh ba cách** trên cùng một chuỗi: vết cặn, quay lui không tĩa, quay lui có tĩa (mục 9.2).
10. **Chạy toàn bộ testcase** ngay trong trang, so với đáp án nhúng sẵn.
11. **Sinh chuỗi ngẫu nhiên có nghiệm** và các test mẫu chọn nhanh.



Hình 2: Giao diện tổng quan.



Hình 3: Chế độ từng bước: dải chữ số tô màu, dòng mã giả đang chạy, nhật ký.



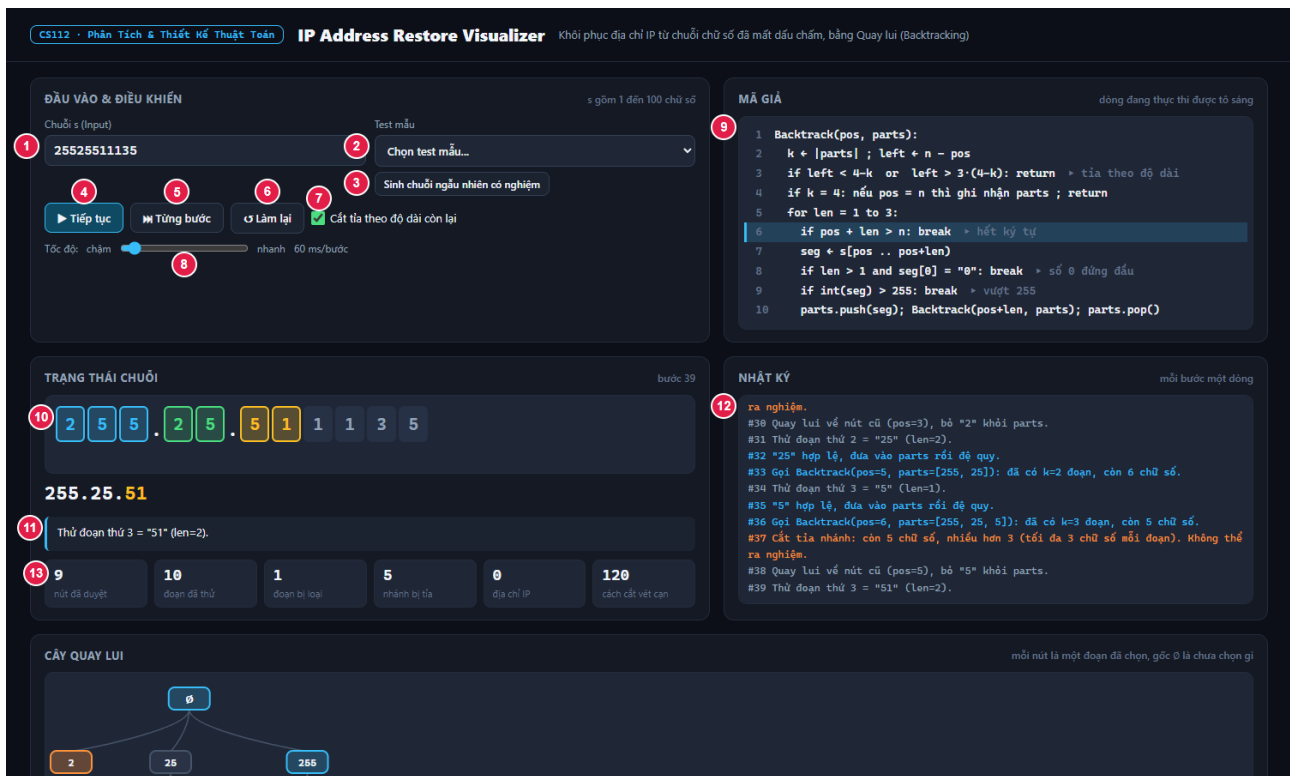
Hình 4: Nhập chuỗi có chữ cái: báo lỗi và khóa nút giải.

7.3 Hướng dẫn sử dụng bảng điều khiển (dashboard)

Cách mở ứng dụng (Windows). Ứng dụng chỉ là một trang web nằm trong một tệp duy nhất, không cần cài đặt gì, không cần Internet.

1. Giải nén tệp `.rar` đã nộp (nhấp chuột phải vào tệp, chọn *Extract Here* hoặc *Giải nén vào đây*). Cần giải nén ra thư mục thường, đừng mở thẳng từ trong tệp nén.
2. Vào thư mục `source` vừa giải nén.
3. **Nhấp đúp vào tệp `index.html`.** Trang sẽ tự mở bằng trình duyệt mặc định (Chrome, Edge hoặc Firefox đều được).
4. Nếu máy mở tệp bằng chương trình khác (ví dụ trình soạn thảo văn bản, hiện ra toàn chữ mã), nhấp chuột phải vào `index.html`, chọn *Open with* (Mở bằng), rồi chọn Chrome hoặc Edge. Hoặc mở trình duyệt trước, rồi kéo thả tệp `index.html` vào cửa sổ trình duyệt.
5. Nếu trang đã mở đúng, sẽ thấy tiêu đề *IP Address Restore Visualizer* nền tối, ô nhập đã điền sẵn `25525511135`. Bấm **Bắt đầu giải** là thấy thuật toán chạy ngay.

Ứng dụng chạy hoàn toàn trên máy: không cần Python, Node.js hay máy chủ. Muốn chạy bản Python nộp WeCode thì dùng lệnh `python source/ip_wecode.py`, nhập một chuỗi rồi nhấn Enter. Các số trong ngoặc tròn bên dưới trùng với số đỏ trên hình 5 và 6.



Hình 5: Nửa trên của bảng điều khiển (ảnh chụp lúc đang chạy dở, đã tạm dừng ở bước 39).

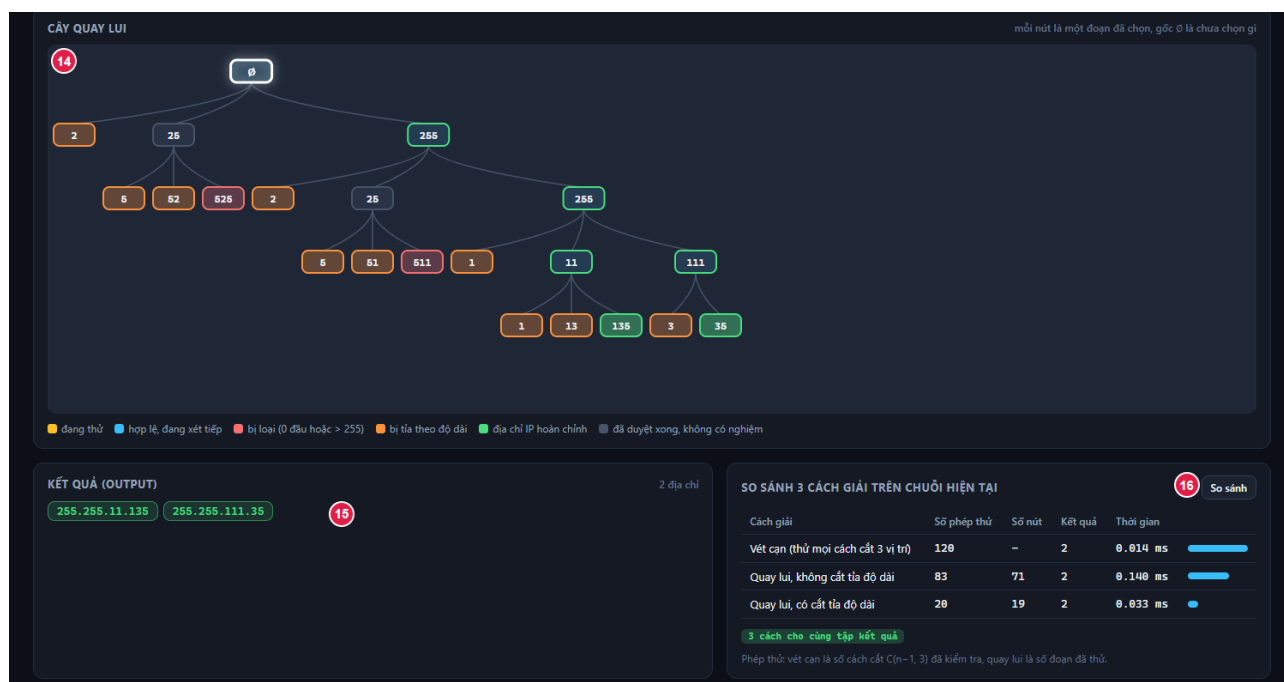
Các bước dùng cơ bản.

1. Nhập chuỗi chữ số vào ô **Chuỗi s** (1), hoặc chọn một test mẫu trong danh sách (2), hoặc bấm nút sinh chuỗi ngẫu nhiên có nghiệm (3). Chuỗi hợp lệ gồm 1 đến 100 chữ số. Nếu nhập rỗng, có chữ cái hoặc dài quá 100, ô nhập báo lỗi màu đỏ và các nút giải bị khóa cho đến khi sửa lại.
2. Bấm **Bắt đầu giải** (4) để thuật toán tự chạy, hoặc **Từng bước** (5) để mỗi lần bấm là một bước. Khi đang chạy, nút (4) đổi thành **Tạm dừng**, sau khi tạm dừng đổi thành **Tiếp tục**, khi chạy xong đổi thành **Chạy lại**.
3. Kéo thanh **Tốc độ** (8) để chỉnh thời gian giữa hai bước, từ 5 đến 800 ms mỗi bước. Muốn xem kỹ thì kéo về phía chậm hoặc dùng nút Từng bước.
4. Bấm **Làm lại** (6) để xóa tiến trình và chạy lại từ đầu với chuỗi hiện tại. Sửa chuỗi trong ô nhập hoặc đổi công tắc (7) cũng tự đặt lại tiến trình.

Đọc các vùng hiển thị.

- **Mã giả** (9): dòng đang thực thi được tô sáng, đối chiếu với mã giả ở mục 5. Dòng 3 có ghi chú “tỉa theo độ dài”, sẽ mờ đi khi tắt công tắc (7).
- **Trạng thái chuỗi** (10): mỗi chữ số một ô. Ô đã thuộc một đoạn được tô màu theo đoạn, ô đang thử có viền vàng (đỏ nếu đoạn bị loại). Dòng chữ lớn bên dưới là địa chỉ IP đang được xây dựng.
- **Dòng trạng thái** (11): một câu mô tả bước vừa thực hiện.
- **Nhật ký** (12): mỗi bước một dòng kèm số thứ tự và lý do. Giữ tối đa 500 dòng gần nhất.

- **Sáu ô thống kê (13):** số nút đã duyệt, số đoạn đã thử, số đoạn bị loại, số nhánh bị tĩa, số địa chỉ IP tìm được, và số cách cắt mà vết cần phải kiểm tra (để so sánh).



Hình 6: Nửa dưới của bảng điều khiển sau khi giải xong chuỗi 25525511135 và bấm So sánh.

- **Cây quay lui (14):** mỗi nút là một đoạn đã chọn, gốc 0 là chưa chọn gì. Màu nút: vàng là đang thử, xanh dương là hợp lệ và đang xét tiếp, đỏ là bị loại (0 đứng đầu hoặc lớn hơn 255), cam là bị tĩa theo độ dài, xanh lá là địa chỉ IP hoàn chỉnh, xám là đã duyệt xong nhưng không có nghiệm. Chú giải màu nằm ngay dưới cây.
 - **Kết quả (Output) (15):** các địa chỉ IP tìm được, thêm dần khi thuật toán ghi nhận nghiệm.
- Các chức năng thêm.**
- **Cắt tỉa theo độ dài còn lại (7):** tắt đi để thấy cây phình ra khi không tĩa, bật lại để so sánh. Đổi công tắc sẽ đặt lại tiến trình.
 - **So sánh (16):** chạy vết cần, quay lui không tĩa và quay lui có tĩa trên chuỗi hiện tại, hiện số phép thử, số nút, số kết quả, thời gian, và báo “3 cách cho cùng tập kết quả” nếu khớp nhau. Thời gian đo trong trình duyệt nên chỉ mang tính tham khảo (cỡ phần nghìn đến phần trăm mili giây với chuỗi ngắn).
 - **Chạy toàn bộ testcase (17):** chạy quay lui có cắt tỉa trên 48 test nhưng sẵn và so với đáp án sinh bởi vết cần. Kết quả hiện ở dòng tổng (18) và bảng chi tiết từng test (hình 7), mỗi dòng có nhóm, tên, độ dài, số đáp án, số thu được, thời gian và nhãn ĐẠT hoặc SAI.

BỘ TESTCASE						
Đạt 48 / 48 testcase		TẤT CẢ ĐẠT	18			
Nhóm	Tên	Độ dài	Đáp án	Thu được	Thời gian	Kết quả
sample	sample_de_bai	11	2	2	0.10 ms	ĐẠT
normal	normal_01	12	0	0	0.10 ms	ĐẠT
normal	normal_02	7	11	11	0.10 ms	ĐẠT
normal	normal_03	6	5	5	0.10 ms	ĐẠT
normal	normal_04	6	9	9	0.00 ms	ĐẠT
normal	normal_05	9	1	1	0.10 ms	ĐẠT
boundary	edge_min_len_4	4	1	1	0.00 ms	ĐẠT
boundary	edge_max_len_12	12	1	1	0.00 ms	ĐẠT

Hình 7: Bảng kết quả chạy toàn bộ testcase: 48/48 đạt.

Một kịch bản thử nhanh (khoảng một phút).

1. Mở trang, giữ nguyên chuỗi 25525511135, bấm **Bắt đầu giải**. Kết quả cuối cùng phải là hai địa chỉ 255.255.11.135 và 255.255.111.35.
2. Bấm **So sánh**. Phải thấy ba cách cùng 2 kết quả, và quay lui có tia dừng ít phép thử nhất.
3. Chọn test mẫu *Có số 0 đứng đầu: 010010*, bấm **Bắt đầu giải**. Phải thấy các nút 01 và 00 màu đỏ (bị loại vì số 0 đứng đầu), và hai kết quả 0.10.0.10, 0.100.1.0.
4. Gõ 12a45. Phải thấy báo lỗi và nút giải bị khóa.
5. Bấm **Chạy toàn bộ testcase**. Phải thấy “Đạt 48 / 48”.

8 Thiết kế thực nghiệm

8.1 Cách tạo testcase

Tập `source/gen_testcases.py` sinh 48 test, cố định hạt giống ngẫu nhiên (giá trị đặt ngay đầu tệp) để tái lập. **Đáp án (.out) không lấy từ lời giải quay lui mà từ ip_bruteforce.py**, thử mọi cách cắt 3 vị trí và kiểm tra từng đoạn. Hai cách giải hoàn toàn khác cấu trúc, nên trùng nhau là bằng chứng độc lập về tính đúng. Dòng trong `.out` được sắp xếp, khi so sánh ta cũng sắp xếp kết quả (để cho phép thử tự bất kỳ).

8.2 Phân nhóm

Nhóm	Số test	Nội dung
sample	1	ví dụ của đề
normal	5	chuỗi thông thường 6 đến 12 ký tự
boundary	9	độ dài 1, 3, 4, 12, 13; toàn 0; toàn 255; đoạn 256
special	7	số 0 đứng đầu, đúng một cách, nhiều đáp án, giá trị sát 255
invalid	2	có chữ cái, chuỗi rỗng
random	20	15 test chữ số ngẫu nhiên, 5 test chữ số {0, 1, 2, 5} để tăng xác suất có nghiệm
large	4	100 chữ số (toàn 1, ngẫu nhiên, toàn 0), 13 chữ số

Tổng cộng 110 địa chỉ đáp án, một test nhiều nhất 16 địa chỉ, 19 test vô nghiệm (đầu vào quá ngắn, quá dài, hoặc không chia được).

8.3 Mức độ bao phủ

Các trường hợp quyết định của thuật toán đều có test riêng: độ dài dưới ngưỡng ($n < 4$), vượt ngưỡng ($n > 12$, tĩa ngay ở gốc), số 0 đứng đầu, đoạn > 255 , đúng 255, một nghiệm duy nhất, nhiều nghiệm. Ngoài 48 test cố định, em còn chạy 3000 chuỗi ngẫu nhiên (độ dài 1 đến 14, bảng chữ số $\{0\}$, $\{0, 1\}$, $\{0, 1, 2, 5\}$, $\{0..9\}$, $\{1, 2\}$) và đối chiếu **ba** cách: quay lui, vét cạn, và module `ipaddress` của Python.

9 Đánh giá

9.1 Tính đúng đắn

- 48/48 testcase đạt, cả trong `test_core.js` (Node.js) lẫn khi bấm “Chạy toàn bộ testcase” trong trang (hình 8).
- 3000 chuỗi ngẫu nhiên: quay lui có tĩa, quay lui không tĩa, vét cạn cho cùng tập kết quả (1140 chuỗi có nghiệm). Bản Python cũng khớp `ipaddress` trên 3000 chuỗi tương tự.
- Ví dụ của đề cho đúng 2 địa chỉ.

BỘ TESTCASE

Đạt 48 / 48 testcase

Tất cả đạt

Chạy toàn bộ testcase

Nhóm	Tên	Độ dài	Đáp án	Thu được	Thời gian	Kết quả
sample	sample_de_bai	11	2	2	0.00 ms	Đạt
normal	normal_01	12	0	0	0.10 ms	Đạt
normal	normal_02	7	11	11	0.20 ms	Đạt
normal	normal_03	6	5	5	0.20 ms	Đạt
normal	normal_04	6	9	9	0.20 ms	Đạt
normal	normal_05	9	1	1	0.10 ms	Đạt
boundary	edge_min_len_4	4	1	1	0.00 ms	Đạt
boundary	edge_max_len_12	12	1	1	0.00 ms	Đạt
boundary	edge_len_13	13	0	0	0.10 ms	Đạt
boundary	edge_len_3	3	0	0	0.00 ms	Đạt
boundary	edge_len_1	1	0	0	0.00 ms	Đạt
boundary	edge_all_zero_4	4	1	1	0.10 ms	Đạt
boundary	edge_all_zero_12	12	0	0	0.00 ms	Đạt
boundary	edge_255_max	12	1	1	0.00 ms	Đạt
boundary	edge_256	12	0	0	0.00 ms	Đạt
special	special_leading_zero_1	6	2	2	0.10 ms	Đạt
special	special_leading_zero_2	4	1	1	0.00 ms	Đạt
special	special_leading_zero_3	5	0	0	0.00 ms	Đạt
special	special_one_way_12	12	1	1	0.10 ms	Đạt
special	special_marry_answers	7	16	16	0.00 ms	Đạt
special	special_digits_10	10	0	0	0.10 ms	Đạt
special	special_255_mixed	10	4	4	0.20 ms	Đạt
invalid	invalid_letters	5	0	0	0.00 ms	Đạt
invalid	invalid_empty	0	0	0	0.00 ms	Đạt
random	random_01	9	0	0	0.40 ms	Đạt
random	random_02	9	0	0	3.60 ms	Đạt
random	random_03	4	1	1	0.10 ms	Đạt
random	random_04	4	1	1	0.00 ms	Đạt
random	random_05	7	6	6	0.30 ms	Đạt
random	random_06	4	1	1	0.00 ms	Đạt
random	random_07	4	1	1	0.10 ms	Đạt
random	random_08	9	0	0	0.10 ms	Đạt
random	random_09	5	4	4	0.00 ms	Đạt
random	random_10	4	1	1	0.00 ms	Đạt
random	random_11	9	0	0	0.10 ms	Đạt
random	random_12	8	6	6	0.10 ms	Đạt
random	random_13	11	2	2	0.10 ms	Đạt
random	random_14	8	3	3	0.10 ms	Đạt
random	random_15	8	5	5	0.20 ms	Đạt
random	random_small_alpha_01	10	5	5	0.10 ms	Đạt
random	random_small_alpha_02	4	1	1	0.00 ms	Đạt
random	random_small_alpha_03	12	0	0	0.00 ms	Đạt
random	random_small_alpha_04	6	5	5	0.20 ms	Đạt
random	random_small_alpha_05	8	12	12	0.20 ms	Đạt
large	large_100_ones	100	0	0	0.00 ms	Đạt
large	large_100_random	100	0	0	0.10 ms	Đạt
large	large_100_zeros	100	0	0	0.00 ms	Đạt
large	large_13_ones	13	0	0	0.00 ms	Đạt

Hình 8: Kết quả chạy toàn bộ testcase trong ứng dụng.

9.2 So sánh ba cách giải

Số liệu do ứng dụng đo (cột “phép thử” là số đoạn đã thử với quay lui, và số cách cắt đã kiểm tra với vét cạn).

Chuỗi	<i>n</i>	IP	Phép thử			Nút (có tia)
			Vét cạn	QL không tia	QL có tia	
25525511135	11	2	120	83	20	19
1921681	7	11	20	60	56	51
010010	6	2	10	16	16	13
0000	4	1	1	7	7	5
255255255255	12	1	165	84	12	13
256256256256	12	0	165	45	3	3
1111111111111	13	0	220	120	0	1
30× 1	30	0	3 654	120	0	1
100× 1	100	0	156 849	120	0	1

Bảng 1: Số phép thử của ba cách giải.

Nhận xét (kể cả điều bất lợi cho quay lui).

- Với chuỗi *dài*, vết cạn tăng như n^3 (156 849 khi $n = 100$) còn quay lui chạm trần 121 nút, và có tia thì chỉ 1 nút, vì tia độ dài phát hiện ngay ở gốc rằng không thể có nghiệm.
- Với chuỗi *rất ngắn*, vết cạn có thể ít phép thử hơn: 010010 (vết cạn 10, quay lui 16), 1921681 (20 so với 56), 0000 (1 so với 7). Lý do: số cách cắt $\binom{n-1}{3}$ nhỏ khi n nhỏ, còn quay lui phải thử cả những đoạn sẽ bị loại. Em coi đây là kết quả thật và ghi lại, không che đi.
- Cắt tia theo độ dài chỉ có tác dụng khi chuỗi gần hoặc vượt các ngưỡng 4 và 12 (ví dụ 255255255255: $84 \rightarrow 12$). Với chuỗi nằm giữa và đều hợp lệ như 1921681 ($60 \rightarrow 56$), lợi ích nhỏ.
- Số phép thử không phải thời gian. Đo lời giải Python 30 lần mỗi chuỗi (tính cả khởi động trình thông dịch, khoảng 28 ms): trung vị 57 ms cho $n = 11$, 73 ms cho $n = 12$, 83 ms cho $n = 100$, tức phần chênh so với khởi động rất nhỏ. Máy đo là Python 3.14 trên WSL, còn bộ kiểm thử mô phỏng bộ chấm chạy bằng Python 3.12.10 trên Windows, thời gian lâu nhất 0,05 s mỗi lần. Chưa có số đo trên máy chấm thật của WeCode.

9.3 Tính trực quan

Mỗi bước thuật toán ứng với đúng một sự kiện trên giao diện: dòng mã giả sáng, dải chữ số đổi màu, nút mới xuất hiện trên cây, một dòng nhật ký. Có thể bấm từng bước để quan sát hoặc để chạy tự động.

9.4 Hạn chế và hướng phát triển

- Thời gian trong bảng so sánh của ứng dụng đo bằng `performance.now()` trong trình duyệt và lặp lại tới 30 ms, chỉ mang tính tham khảo (dao động giữa các lần chạy).
- Ứng dụng chỉ hỗ trợ IPv4 và chuỗi chữ số thập phân.
- Hướng phát triển: cho phép đặt khoảng giá trị khác 0–255, hỗ trợ IPv6, hoặc đếm số nghiệm bằng quy hoạch động để so sánh với quay lui.

10 Quy trình thực hiện với AI

10.1 Công cụ

- **Jcode** (trợ lý AI chạy trong terminal, có quyền đọc/ghi file và chạy lệnh), dùng một mô hình duy nhất là **Claude Sonnet 5.5** (Anthropic). Mọi công việc AI hỗ trợ trong bài này (viết code, kiểm thử, viết báo cáo) đều do mô hình này thực hiện, em không dùng mô hình ngôn ngữ nào khác.
- **Giọng đọc Ngọc Huyền** của engine **VieNeu-TTS** (model **VieNeu-TTS-v3-Turbo**), chạy cục bộ bằng Python đi kèm ứng dụng Reup-Video trên máy em.
- **Playwright** (điều khiển Chrome) và **ffmpeg** để quay và ghép video.
- **Tectonic** (bộ biên dịch LaTeX) để tạo PDF của báo cáo này.

10.2 Phân công

- **Chọn bài:** em chọn bài Địa chỉ IP vì bài có liên quan tới công việc của em, và dùng quay lui theo đúng yêu cầu của đề.
- **AI đảm nhiệm:** viết lời giải Python, bản vét cạn, bộ sinh testcase, ứng dụng web, kịch bản video, bản nháp báo cáo này.
- **Em đảm nhiệm:**
 - cung cấp đầu vào cho AI: yêu cầu của cô về bài tập lớn và các hình ảnh chụp liên quan tới đề tài em muốn làm (đề bài WeCode);
 - chọn giọng đọc Ngọc Huyền cho video;
 - xem lại kết quả của phần demo đã xây dựng;
 - đọc lại báo cáo và điều chỉnh cho hợp lý, đầy đủ hơn;
 - nghe lại video để kiểm tra có bị mất âm không và phụ đề có khớp với giao diện demo không.

10.3 Các bước và yêu cầu chính

#	Yêu cầu (tóm tắt)	Kết quả
1	Em đưa đường dẫn đề bài WeCode và thư mục bài Dò Mìn cũ (để tham khảo cách tổ chức), nhờ dựng bài “dãy IP” có lồng giọng Ngọc Huyền.	AI kiểm tra giọng Ngọc Huyền chạy được trên máy, rồi nhờ em gửi đề bài để làm đúng đề.
2	Em gửi ảnh đề bài “Địa chỉ IP”.	Xác định đề là “Địa chỉ IP”, bắt buộc quay lui.
3	“Làm từng cái một”.	AI viết lời giải quay lui, bản vét cạn độc lập, bộ 48 testcase và đối chứng 3000 chuỗi ngẫu nhiên.
4	Dựng ứng dụng trực quan hóa.	AI viết <code>index.html</code> , tự kiểm thử bằng Node.js và Chrome thật.
5	Làm video có giọng Ngọc Huyền.	AI viết kịch bản 11 cảnh, sinh giọng đọc, quay bằng Playwright, ghép bằng ffmpeg.
6	Viết báo cáo theo đề cô.	AI viết báo cáo này, lấy số liệu thật từ ứng dụng.
7	Rà soát lại toàn bộ bằng chứng đã có, sửa chỗ nói quá.	AI lập lại todo, đối chiếu từng mục với bằng chứng, phát hiện 3 tham chiếu mục trong báo cáo bị ghi sai số (ghi cứng, lệch so với số mục thật), và độ dài video đã lỗi thời sau khi quay lại (2 phút 54 giây xuống 2 phút 50 giây, và sau đó là 3 phút 9 giây khi thêm cảnh giới thiệu các tùy chọn), sửa và biên dịch lại.
8	Em đọc lại báo cáo và yêu cầu chỉnh các phần mô hình AI, lý do chọn bài, phần em đảm nhiệm, nhận xét cá nhân.	AI sửa theo yêu cầu, thêm mẫu prompt kiểm thử bên dưới.
9	Rà soát điều kiện chấp nhận: kiểm tra từng yêu cầu của đề, chạy lời giải như bộ chấm, kiểm thử toàn bộ giao diện trên Chrome thật.	AI chạy 192 lần lời giải (kiểm tra bốn dạng đầu vào) và 28 kiểm tra giao diện, đều đạt. Phát hiện video chưa giới thiệu 3 chức năng của ứng dụng (công tắc cắt tia, nút sinh chuỗi ngẫu nhiên, danh sách test mẫu) trong khi đề yêu cầu giới thiệu đầy đủ chức năng, nên thêm cảnh thứ 7 và quay lại.

Mẫu prompt kiểm thử đề xuất. Đây là mẫu prompt em đề xuất dùng ngay sau khi đã cung cấp đủ đầu vào (yêu cầu của cô, đề bài, hình ảnh), để AI tự kiểm thử trước khi báo hoàn thành:

“Sau khi đã có đủ đầu vào, hãy lập plan và todo list cho toàn bộ công việc. Kiểm tra từng bước ngay khi làm xong bước đó. Viết testcase cho thuật toán và đối chiếu với một cách giải độc lập. Chạy end-to-end test trên giao diện thật (nhập input, chạy, xem output). Chỉ báo hoàn thành khi có bằng chứng (lệnh đã chạy, kết quả thấy được), và nói rõ phần nào chưa kiểm chứng được. Nếu thiếu đầu vào thì hỏi lại, không đoán.”

Sau khi AI báo xong, em vẫn phải tự kiểm thử lại bằng tay (manual test) trước khi tin kết quả.

10.4 Lỗi gặp phải và cách phát hiện, sửa

Những lỗi dưới đây có thật, xảy ra trong quá trình làm bài này (lỗi 6 và 7 là lỗi của bản nháp báo cáo và video, các lỗi còn lại là lỗi môi trường và công cụ).

1. **Cây quay lui bị cắt hàng nút cuối.** Khung vẽ cao 340 px không đủ cho cây 5 tầng. *Phát hiện:* AI xem ảnh chụp giao diện. *Sửa:* tăng lên 430 px.
2. **Lỗi lúc kiểm thử.** Harness Node.js khai báo trùng tên `runAll` (`SyntaxError`). *Sửa:* chạy lỗi thuật toán trong một `Function` riêng.
3. **In tiếng Việt ra console Windows.** Python in lỗi `UnicodeEncodeError` vì console dùng bảng mã cp1252. *Sửa:* `sys.stdout.reconfigure(encoding='utf-8')`.
4. **Quay video báo thiếu ffmpeg.** Playwright cần bản ffmpeg riêng để ghi video. *Sửa:* chạy `playwright install ffmpeg`.
5. **Chromium của Playwright không chạy trên WSL** (thiếu thư viện `libnspr4`). *Sửa:* chuyển sang dùng Chrome thật của Windows (`channel='chrome'`).
6. **Số liệu trong báo cáo.** Bản nháp đầu của bảng ví dụ minh họa có lý do tĩa ghi sai (ví dụ nút 25.5 bị ghi “còn 9 ký tự” trong khi thực tế còn 8) và có ghi chú tự mâu thuẫn. *Phát hiện:* AI tự đọc lại bảng và tính lại. *Sửa:* xuất đúng cây quay lui từ ứng dụng, tính lại k , ℓ và khoảng cần có cho từng nút bằng mã, rồi thay toàn bộ bảng (mục 6 và phụ lục).
7. **Video thiếu chức năng.** Bản video đầu chưa giới thiệu công tắc cắt tia, nút sinh chuỗi ngẫu nhiên và danh sách test mẫu, dù đề yêu cầu giới thiệu đầy đủ chức năng. *Phát hiện:* AI đối chiếu từng chức năng của ứng dụng với các thao tác trong video. *Sửa:* thêm cảnh giới thiệu các tùy chọn và quay lại (video 3 phút 9 giây).

10.5 Kiểm chứng kết quả: ai làm gì

Em tách rõ phần AI tự kiểm chứng và phần em kiểm tra, vì hai việc này khác nhau.

AI thực hiện (em chỉ yêu cầu, không tự chạy):

- Không tin kết quả khi chưa có bằng chứng: đáp án testcase sinh bằng vết cặn, một thuật toán khác hẳn, rồi đối chiếu với chính lời giải quay lui.
- Chạy ba cách giải trên 3000 chuỗi ngẫu nhiên, thêm module chuẩn `ipaddress` của Python làm cách thứ tư độc lập.

- Xem ảnh chụp giao diện và khung hình video, không chỉ dựa vào đầu ra của script.

Em thực hiện:

- Xem lại hình ảnh output thông qua video demo: đối chiếu kết quả hiển thị trên giao diện (hai địa chỉ của ví dụ, bảng thống kê, kết quả 48/48 testcase) với đáp án mẫu của đề.
- Đọc lại báo cáo, yêu cầu AI chỉnh các phần chưa hợp lý (mô hình AI, lý do chọn bài, phần em đảm nhiệm, nhận xét cá nhân).
- Nghe lại video để kiểm tra mắt âm và phụ đề có khớp giao diện demo không, rồi tự kiểm thử lại bằng tay trước khi nộp.

10.6 Nhận xét cá nhân

AI chỉ thực sự làm tốt khi đầu vào (input) được cung cấp chuẩn và đầy đủ: yêu cầu của cô, đề bài, hình ảnh và danh sách công việc phải thực hiện. Khi đầu vào thiếu, AI dễ suy diễn rồi đưa ra kết quả sai nhưng trình bày rất tự tin, đó là hiện tượng ảo giác (hallucination). Trong bài này có hai ví dụ cụ thể: ở bản nháp đầu, bảng ví dụ minh họa trong báo cáo có lý do tả ghi sai, chỉ phát hiện được khi tính lại từng nút bằng mã; và bản video đầu thiếu ba chức năng của ứng dụng mà không ai nhắc, chỉ phát hiện khi đối chiếu từng chức năng với kịch bản.

Vì vậy quy trình em rút ra là:

1. cung cấp đầu vào chuẩn và đủ;
2. yêu cầu AI lập plan và todo list cho các công việc phải thực hiện;
3. kiểm tra từng bước, viết testcase và đối chiếu với một cách giải độc lập;
4. chạy end-to-end test khi làm xong (mẫu prompt ở mục 10.3);
5. cuối cùng bản thân em phải kiểm thử lại bằng tay (manual test), không tin hoàn toàn vào báo cáo của AI.

11 Tài liệu tham khảo

Phần đề cung cấp cho AI không nêu luật địa chỉ IP, nên phần kiến thức này đến từ AI và được đối chiếu với các nguồn dưới đây (AI tra cứu ngày 07/10/2026 để kiểm tra, không phải nguồn dùng để viết lần đầu).

1. **WeCode, Lab#01 (Backtracking), bài “Địa chỉ IP”.** Đề của môn. Nguồn của tên bài, yêu cầu quay lui và ví dụ 25525511135, lấy từ ảnh chụp đề em cung cấp.
2. **LeetCode, 93. Restore IP Addresses.** <https://leetcode.com/problems/restore-ip-addresses/>. Bài cùng dạng. Trang này chặn công cụ tải tự động, nên nội dung đề được đối chiếu qua bản chép trong kho github.com/doocs/leetcode trên GitHub (thư mục lời giải bài 93). *Xác nhận:* luật mỗi đoạn từ 0 đến 255, không có số 0 đứng đầu, giữ nguyên thứ tự chữ số, và các ví dụ 25525511135, 0000. *Khác đề của em:* LeetCode cho phép in theo thứ tự bất kỳ và giới hạn độ dài là 20, còn ứng dụng này nhận đến 100 ký tự.
3. **RFC 3986, mục 3.2.2 (cú pháp IPv4address).** <https://www.rfc-editor.org/rfc/rfc3986>. Văn phạm dec-octet cho mỗi đoạn là một chữ số, hoặc hai chữ số không bắt đầu bằng 0, hoặc ba chữ số có giá trị đến 255. Điều này khớp với bốn ràng buộc ở mục 4.

4. **RFC 791, Internet Protocol (1981).** <https://www.rfc-editor.org/rfc/rfc791>.
Xác nhận địa chỉ IPv4 dài cố định bốn octet (32 bit), tức bốn đoạn mỗi đoạn tối đa 255.
RFC này không bàn đến cách viết thập phân có chấm, nên dạng viết “không số 0 đứng đầu” dựa vào RFC 3986 và LeetCode ở trên.

Chưa có nguồn xác nhận. Hai điểm sau là lựa chọn của em, không có nguồn nào xác nhận cho đề WeCode: (a) thứ tự in kết quả, và (b) in rỗng khi không có địa chỉ nào hoặc đầu vào có chữ. Nếu bộ chấm của WeCode yêu cầu khác thì phải sửa lời giải. Cách cắt tỉa theo độ dài còn lại cũng gặp trong các lời giải quay lui công bố cho bài LeetCode 93, nhưng em không sao chép từ nguồn cụ thể nào, nên không ghi như một tham chiếu.

12 Kết luận

Em đã xây dựng ứng dụng web trực quan hóa thuật toán quay lui cho bài Địa chỉ IP: nhập chuỗi, theo dõi từng bước trên dải chữ số, mã giả, nhật ký và cây quay lui, rồi xem kết quả. Thuật toán đúng trên 48 testcase và 3000 chuỗi ngẫu nhiên, đối chiếu với vét cạn. Phần so sánh cho thấy quay lui có cắt tỉa thắng rõ khi chuỗi dài (1 nút so với 156 849 cách cắt cho $n = 100$), nhưng không phải lúc nào cũng thắng: với chuỗi ngắn, vét cạn thử ít hơn.

A Cây quay lui đầy đủ của 25525511135

Xuất từ ứng dụng (runAll, bật tỉa). Cột 1 là mã nút, cột 2 là độ sâu.

1	1	2	tia do dai
2	1	25	di tiep, khong co nghiem
3	2	25.5	tia do dai
4	2	25.52	tia do dai
5	2	25.525	loai (>255)
6	1	255	di tiep, co nghiem
7	2	255.2	tia do dai
8	2	255.25	di tiep, khong co nghiem
9	3	255.25.5	tia do dai
10	3	255.25.51	tia do dai
11	3	255.25.511	loai (>255)
12	2	255.255	di tiep, co nghiem
13	3	255.255.1	tia do dai
14	3	255.255.11	di tiep, co nghiem
15	4	255.255.11.1	tia do dai
16	4	255.255.11.13	tia do dai
17	4	255.255.11.135	NGHIEM
18	3	255.255.111	di tiep, co nghiem
19	4	255.255.111.3	tia do dai
20	4	255.255.111.35	NGHIEM

Thống kê: 21 nút (kể cả gốc), 19 nút được duyệt, 20 đoạn đã thử, 2 đoạn bị loại, 10 nhánh bị tỉa, 2 nghiệm.